Practical – 7

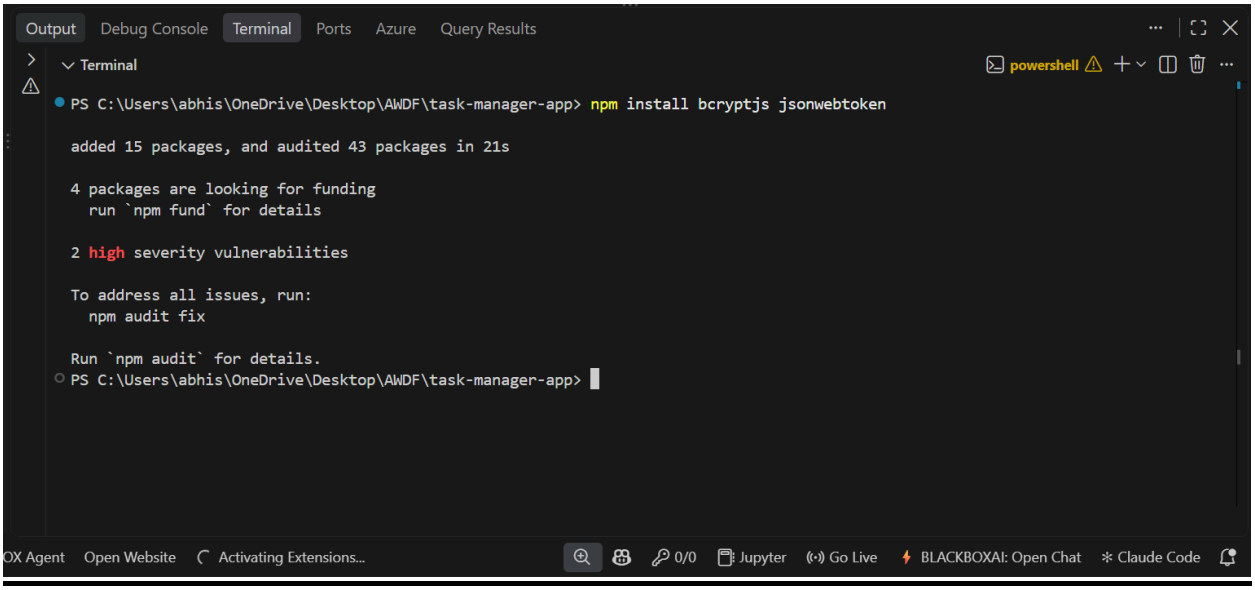
Aim:

To implement JWT-based authentication and input validation as part of the Express middleware pipeline.

Content:

After completing this practical, I will be able to implement JWT-based authentication and input validation in a full-stack task management application using Node.js, Express.js, MongoDB, and Mongoose. I will learn to create user registration and login functionality, securely hash passwords using bcrypt, generate and verify JWT tokens, protect routes using authentication middleware, and validate user input on the server. This practical will help me understand secure authentication and develop reliable and secure backend applications.

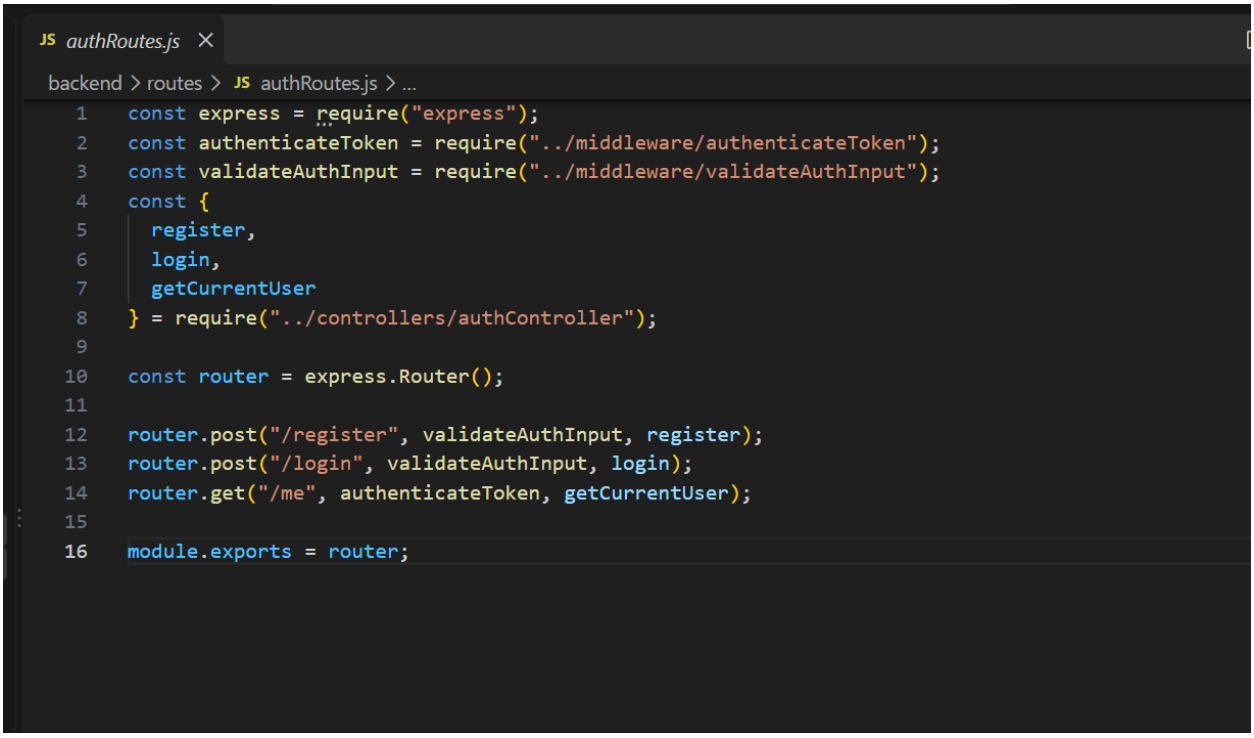
Screen Shot:



JS User.js

backend > models > JS User.js > ...

```
1   const mongoose = require("mongoose");
2
3   const userSchema = new mongoose.Schema(
4     {
5       email: {
6         type: String,
7         required: true,
8         unique: true,
9         lowercase: true,
10        trim: true
11      },
12      password: {
13        type: String,
14        required: true,
15        select: false
16      }
17    },
18    { timestamps: true }
19  );
20
21  module.exports = mongoose.model("User", userSchema);
```


authRoutes.js
backend > routes > authRoutes.js > ...
1 const express = require("express");
2 const authenticateToken = require("../middleware/authenticateToken");
3 const validateAuthInput = require("../middleware/validateAuthInput");
4 const {
5 register,
6 login,
7 getCurrentUser
8 } = require("../controllers/authController");
9
10 const router = express.Router();
11
12 router.post("/register", validateAuthInput, register);
13 router.post("/login", validateAuthInput, login);
14 router.get("/me", authenticateToken, getCurrentUser);
15
16 module.exports = router;

authController.js

backend > controllers > authController.js > register > register

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const publicUser = (user) => ({
  id: user._id,
  email: user.email
});

exports.register = async (req, res, next) => {
  try {
    const email = req.body.email.trim().toLowerCase();
    const existingUser = await User.findOne({ email });

    if (existingUser) {
      return res.status(409).json({
        success: false,
        error: "An account with this email already exists"
      });
    }

    const password = await bcrypt.hash(req.body.password, 10);
    const user = await User.create({ email, password });

    res.status(201).json({
      success: true,
      data: { user: publicUser(user) }
    });
  } catch (err) {
    if (err.code === 11000) {
      return res.status(409).json({
        success: false,
        error: "An account with this email already exists"
      });
    }
    next(err);
  }
};

```

```
exports.login = async (req, res, next) => {
  try {
    const email = req.body.email.trim().toLowerCase();
    const user = await User.findOne({ email }).select("+password");

    if (!user || !(await bcrypt.compare(req.body.password, user.password))) {
      return res.status(401).json({
        success: false,
        error: "Email or password is incorrect"
      });
    }

    const token = jwt.sign(
      { id: user._id.toString() },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );

    res.status(200).json({
      success: true,
      data: { token, user: publicUser(user) }
    });
  } catch (err) {
    next(err);
  }
};
```

authenticateToken.js

backend > middleware > authenticateToken.js > ...

```
const jwt = require("jsonwebtoken");

const authenticateToken = (req, res, next) => {
  const authorization = req.headers.authorization || "";
  const [scheme, token] = authorization.split(" ");

  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({
      success: false,
      error: "Authentication required. Provide a Bearer token."
    });
  }

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({
      success: false,
      error: err.name === "TokenExpiredError" ? "Token has expired" : "Invalid token"
    });
  }
};

module.exports = authenticateToken;
```

validateAuthInput.js

backend > middleware > validateAuthInput.js > validateAuthInput

```
const validateAuthInput = (req, res, next) => {
  const { email, password } = req.body || {};

  if (typeof email !== "string" || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.trim())) {
    return res.status(400).json({
      success: false,
      error: "A valid email address is required"
    });
  }

  if (typeof password !== "string" || password.length < 8 || password.length > 72) {
    return res.status(400).json({
      success: false,
      error: "Password must be between 8 and 72 characters"
    });
  }

  next();
};

module.exports = validateAuthInput;
```

taskRoutes.js

backend > routes > taskRoutes.js > ...

```
const express = require("express");
const router = express.Router();

const validateTaskId = require("../middleware/validateTaskId");
const authenticateToken = require("../middleware/authenticateToken");
const validateTaskInput = require("../middleware/validateTaskInput");
const {
  getAllTasks,
  getTaskById,
  createTask,
  updateTask,
  deleteTask
} = require("../controllers/taskController");

router.use(authenticateToken);

router.get("/", getAllTasks);
router.get("/:id", validateTaskId, getTaskById);
router.post("/", validateTaskInput(), createTask);
router.put("/:id", validateTaskId, validateTaskInput(true), updateTask);
router.delete("/:id", validateTaskId, deleteTask);

module.exports = router;

```

validateTaskInput.js

backend > middleware > validateTaskInput.js > validateTaskInput > <function>

```
const VALID_STATUSES = ["pending", "in-progress", "done"];
const VALID_PRIORITIES = ["low", "medium", "high"];

const validateTaskInput = (partial = false) => (req, res, next) => {
  const body = req.body;

  if (!body || typeof body !== "object" || Array.isArray(body)) {
    return res.status(400).json({ success: false, error: "A JSON object is required" });
  }

  if (!partial && (typeof body.title !== "string" || !body.title.trim())) {
    return res.status(400).json({
      success: false,
      error: "Title is required and must be a non-empty string"
    });
  }

  if (body.title !== undefined && (typeof body.title !== "string" || !body.title.trim())) {
    return res.status(400).json({
      success: false,
      error: "Title must be a non-empty string"
    });
  }

  if (body.description !== undefined && typeof body.description !== "string") {
    return res.status(400).json({ success: false, error: "Description must be a string" });
  }

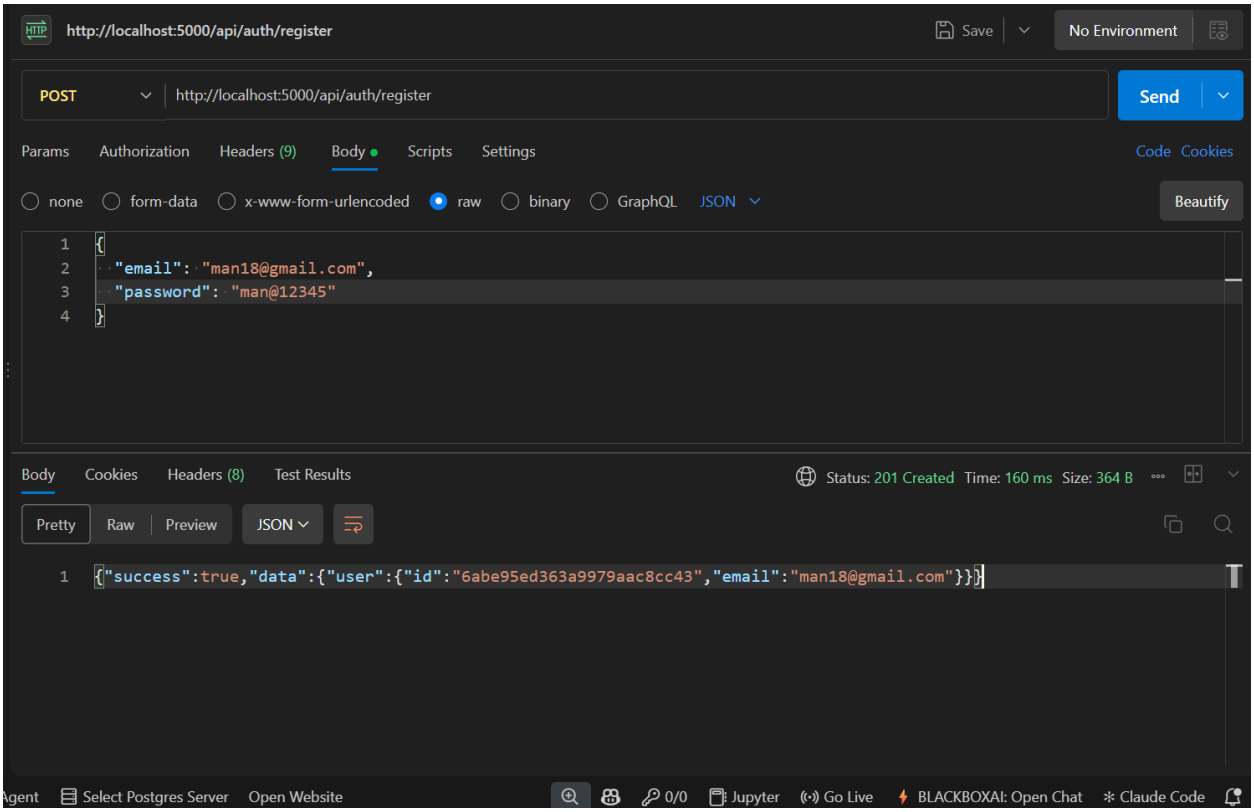
  if (body.status !== undefined && !VALID_STATUSES.includes(body.status)) {
    return res.status(400).json({
      success: false,
      error: `Status must be one of: ${VALID_STATUSES.join(", ")}`
    });
  }

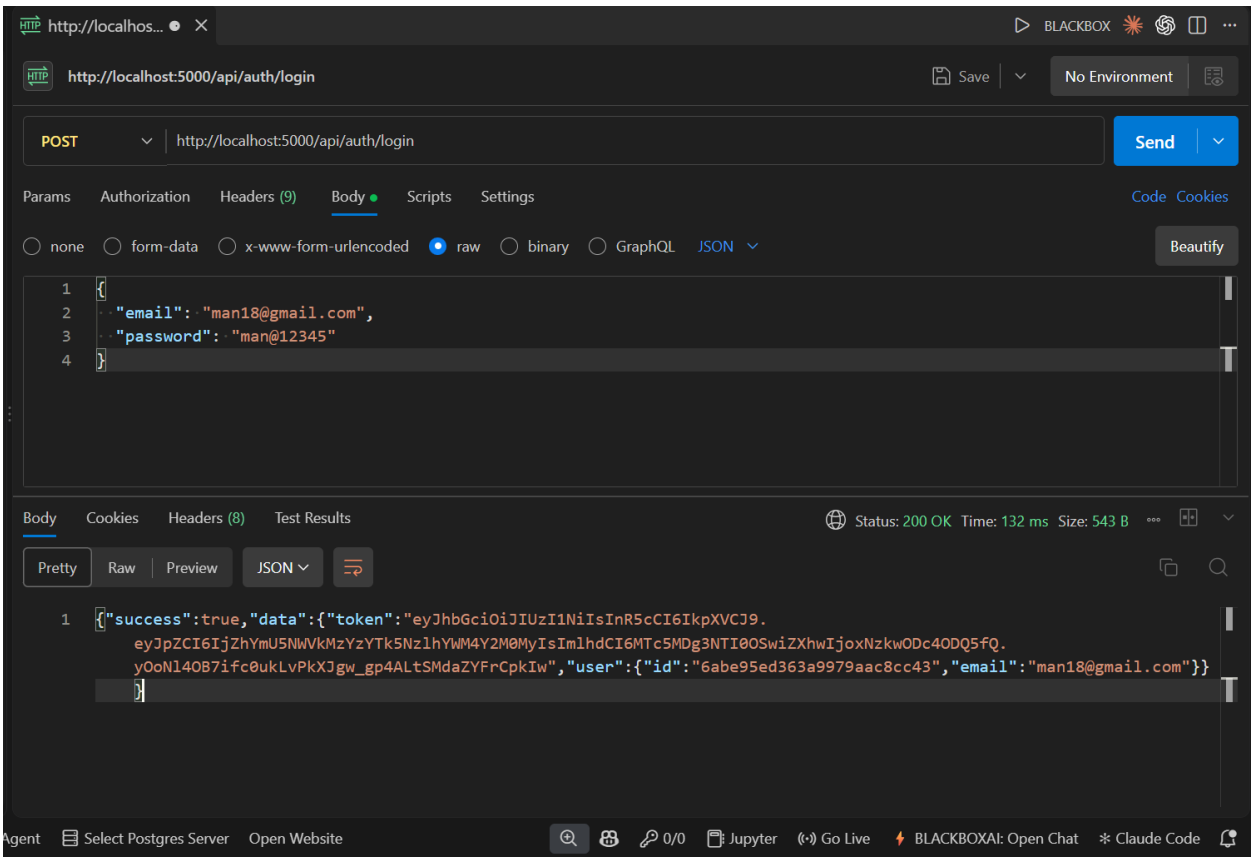
  if (body.priority !== undefined && !VALID_PRIORITIES.includes(body.priority)) {
    return res.status(400).json({
      success: false,
      error: `Priority must be one of: ${VALID_PRIORITIES.join(", ")}`
    });
  }

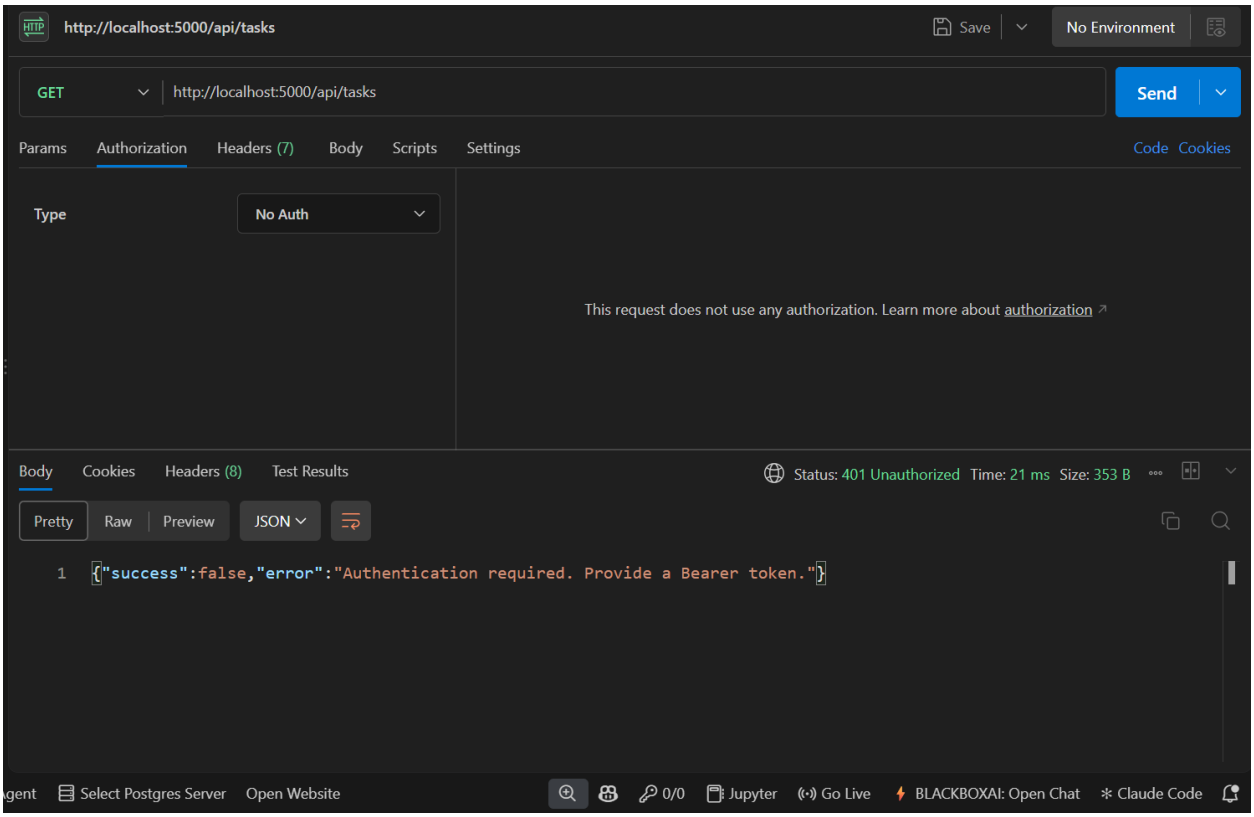
  if (body.completed !== undefined && typeof body.completed !== "boolean") {
    return res.status(400).json({ success: false, error: "Completed must be a boolean" });
  }

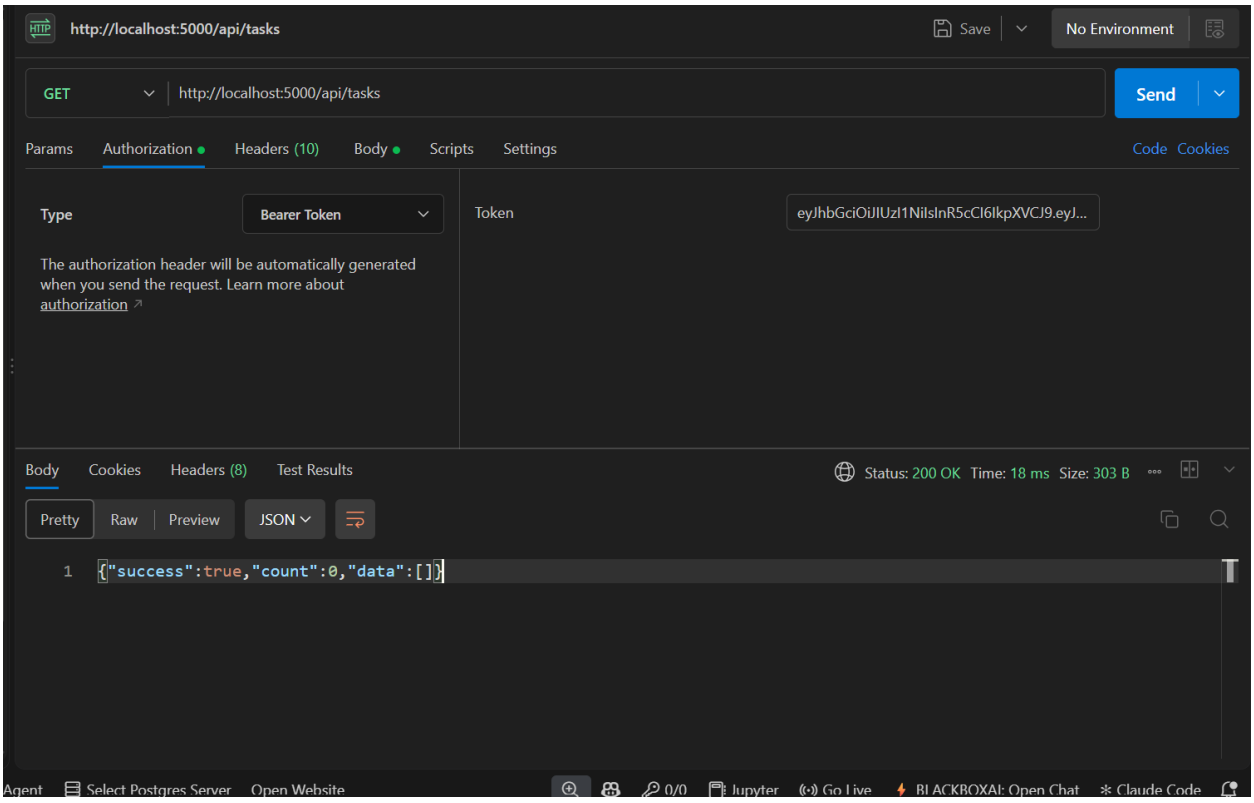
  next();
};
```

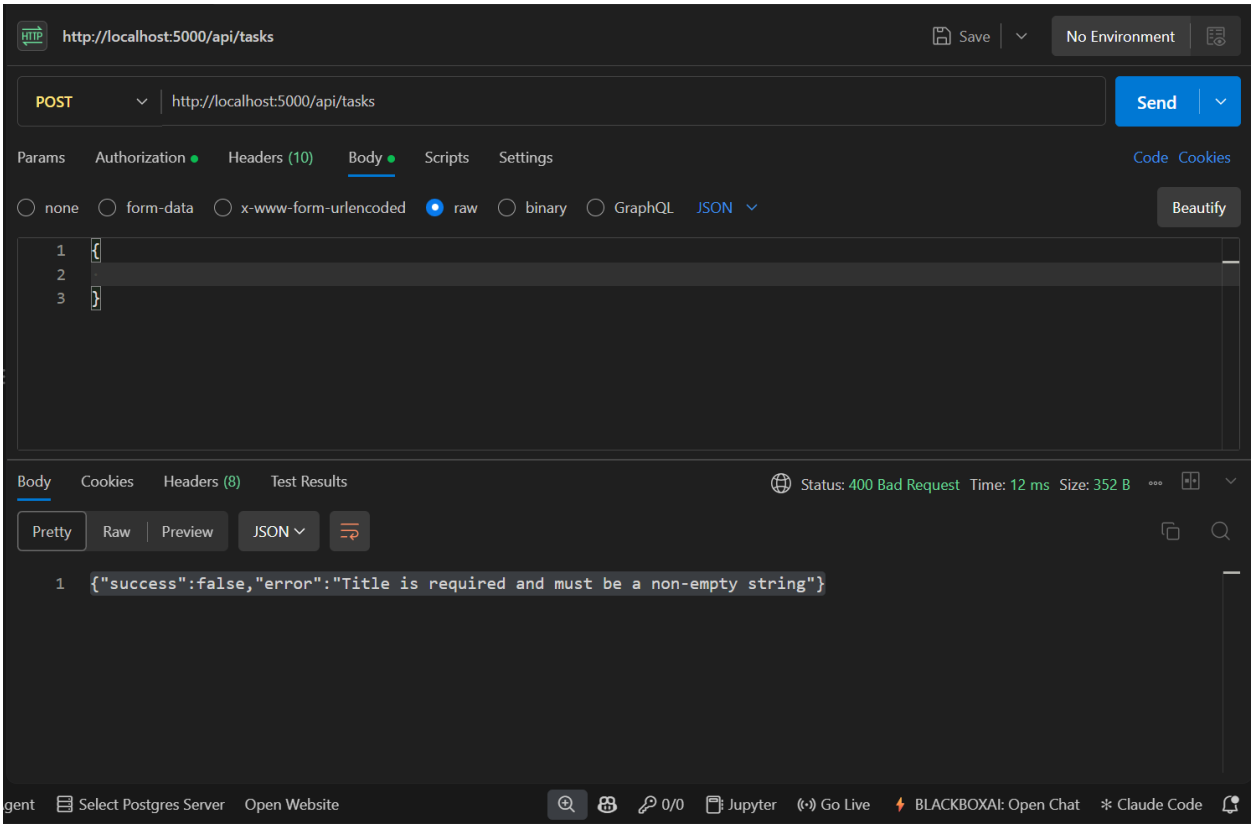
OX Agent Select Postgres Server Open Website Ln 36, Col 73 Spaces: 2 UTF

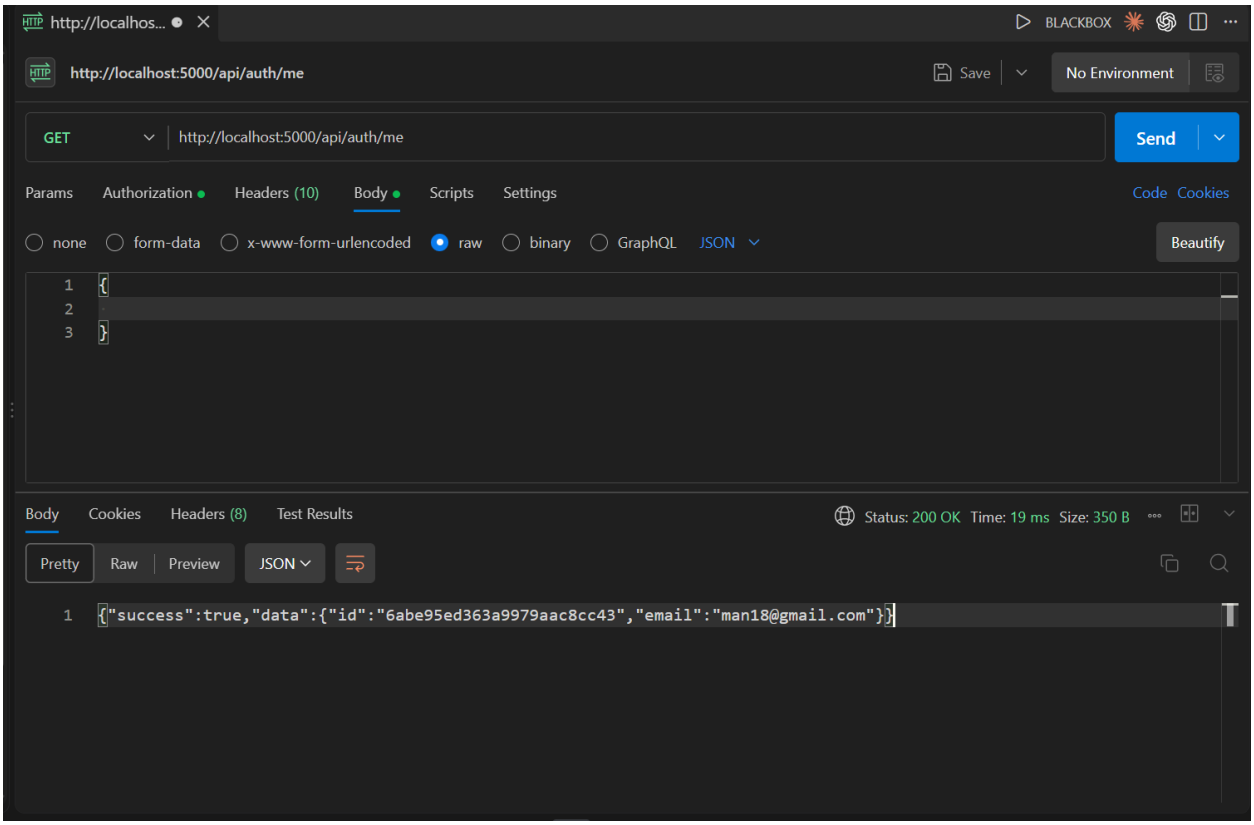

http://localhost:5000/api/auth/register
Save
No Environment
POST
http://localhost:5000/api/auth/register
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"email": "man18@gmail.com",
"password": "man@12345"
}
Body
Cookies
Headers (8)
Test Results
Status: 201 Created
Time: 160 ms
Size: 364 B
Pretty
Raw
Preview
JSON
{"success":true,"data":{"user":{"id":"6abe95ed363a9979aac8cc43","email":"man18@gmail.com"}}}
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhos...
BLACKBOX
http://localhost:5000/api/auth/login
Save
No Environment
POST
http://localhost:5000/api/auth/login
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"email": "man18@gmail.com",
"password": "man@12345"
}
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 132 ms
Size: 543 B
Pretty
Raw
Preview
JSON
{"success":true,"data":{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhYmU5NWVkMzYzYTk5NzlhYWM4Y2M0MyIsImlhdCI6MTc5MDg3NTI0OSwiZXhwIjoxNzkwODc4ODQ5fQ.
yOoNl4OB7ifc0ukLvPkXJgw_gp4ALtSMdaZYFrCpkIw","user":{"id":"6abe95ed363a9979aac8cc43","email":"man18@gmail.com"}}
}
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhost:5000/api/tasks
Save
No Environment
GET
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (7)
Body
Scripts
Settings
Code
Cookies
Type
No Auth
This request does not use any authorization. Learn more about authorization
Body
Cookies
Headers (8)
Test Results
Status: 401 Unauthorized
Time: 21 ms
Size: 353 B
Pretty
Raw
Preview
JSON
1 {"success":false,"error":"Authentication required. Provide a Bearer token."}
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhost:5000/api/tasks
Save
No Environment
GET
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
Type
Bearer Token
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 18 ms
Size: 303 B
Pretty
Raw
Preview
JSON
1 {"success":true,"count":0,"data":[]}
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhost:5000/api/tasks
Save
No Environment
POST
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
}
Body
Cookies
Headers (8)
Test Results
Status: 400 Bad Request
Time: 12 ms
Size: 352 B
Pretty
Raw
Preview
JSON
{"success":false,"error":"Title is required and must be a non-empty string"}
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhos...
BLACKBOX
http://localhost:5000/api/auth/me
Save
No Environment
GET
http://localhost:5000/api/auth/me
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
}
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 19 ms
Size: 350 B
Pretty
Raw
Preview
JSON
{"success":true,"data":{"id":"6abe95ed363a9979aac8cc43","email":"man18@gmail.com"}}

Conclusion:

This practical demonstrates the implementation of JWT-based authentication and input validation in a full-stack Task Management application using Node.js, Express.js, MongoDB, and Mongoose. It includes user registration and login functionality, secure password hashing using bcrypt, JWT token generation and verification, and protected routes using authentication middleware. It also covers server-side input validation, error handling, and testing of authentication and authorization through Postman.